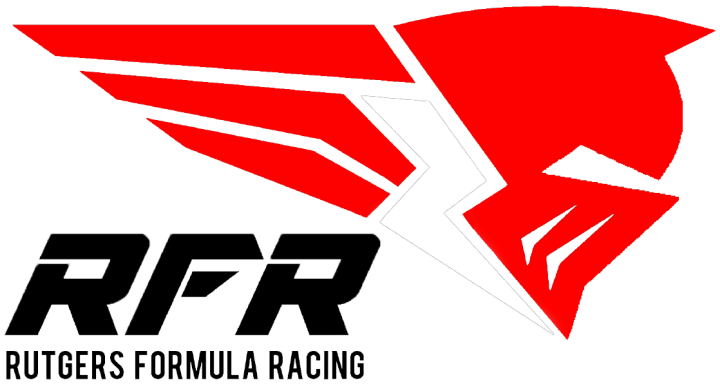
Temperature Monitor Design Report

Rutgers Formula Racing

Shardul Kulkarni | Paul Rubinshteyn | Ryan Catacutan
November 3, 2026



Contents

1 Revision Log

2 Purpose

3 Parties Involved

4 Research

4.1 Requirements

4.2 Why Do We Need This?

4.3 Pros & Cons

4.4 Design Impact

5 Notes/Findings

5.1 Core Requirements

5.2 How the Sensing Works

5.3 Part Choices and Justification

5.4 Schematic

5.4.1 Pin Map

5.5 Workflow

5.6 Firmware

5.6.1 Pins and Constants

5.6.2 setup()

5.6.3 Reading the Channels

5.6.4 Sending the Data and the Fault LED

5.6.5 Timing

5.7 Limitations and Recommendations

6 Conclusion

1 Revision Log

Editor	Date	Revision / Change
Dennis Kahraman	Sep. 24, 2026	1 — Initial repository commit (fa919a4)
Paul Rubinshteyn	Oct. 14, 2026	2 — Uploaded project files; renamed Power.pdf to SchematicVersion1.pdf (e808912, 8054aa1)
Shardul Kulkarni	Oct. 15, 2026	3 — Added hardware schematic flowchart v1 (338aa98)
Ryan Catacutan	Oct. 19, 2026	4 — Added temperature monitor code (0d95c6c)
Rdek113	Oct. 22, 2026	5 — Uploaded additional files (a9974fe)
Paul Rubinshteyn	Nov. 3, 2026	6 — Five file-upload commits (38e0ad9, 27489a5, 6d23ba0, 4b0011e, ca079de)
Shardul Kulkarni	Nov. 3, 2026	7 — Updated design report revision log
Ryan Catacutan	Nov. 4, 2026	8 — Linear interpolation for temp conversion; updated Section 5.6

2 Purpose

This report walks through the design of a 16-channel temperature monitor board: the schematic, how signals move through it, and the firmware that runs it. The board reads sixteen diode temperature sensors through one analog multiplexer, sends every reading out over I²C, and blinks a fault LED if anything goes above 60 °C.

3 Parties Involved

Name	Team Role	Report Role
Shardul Kulkarni	Member	Co-author
Paul Rubinshteyn	Member	Co-author
Ryan Catacutan	Member	Co-author

4 Research

4.1 Requirements

Here is what the board has to do, based on how the schematic and firmware are built.

- **Supply** Run from a 12 V input (P1) and make its own 5 V rail. That rail powers the microcontroller, the multiplexer, the sensor bias resistors, and the I²C pull-ups.
- **Sensing** Measure 16 channels (CH0 to CH15), one diode sensor per channel, using a single ADC input (A0).
- **Reporting** Every cycle, write one I²C message to address 0x48:
  (a) the maximum temperature across all 16 channels, sent first
  (b) the 16 individual channel temperatures, in order from CH0 to CH15
- **Fault indication** Blink the fault LED (D7 through a 330 Ω resistor) while any channel is above 60 °C, and keep it off otherwise.

4.2 Why Do We Need This?

When you have a bunch of places that can heat up on their own, one temperature reading doesn't tell you much. You need to know which one is the problem. This board checks 16 points at once with a single analog input, reports every reading plus the maximum over I²C so a host can log the data or react to the worst case, and runs its own fault LED so a problem is still obvious even if nothing is reading the bus.

4.3 Pros & Cons

Pros

1. One ADC pin reads all 16 sensors, so the multiplexer saves 15 analog inputs on the microcontroller
2. The sensors are regular small-signal diodes, which are cheap and only need one resistor each
3. The maximum temperature is sent first, so a host that only cares about the worst case can read one byte
4. The fault LED works on its own, with or without the I²C bus
5. The firmware is one short sketch that is easy to read and change

Cons

1. The 10-bit ADC limits resolution to about 2.4 °C per count
2. Absolute accuracy depends on the assumed 0.6 V diode drop at 25 °C, which is a little different for every diode
3. An open or shorted sensor does not trigger a fault
4. Each temperature is sent as one byte, so values outside the signed 8-bit range wrap around

4.4 Design Impact

The board needs one 12 V connection (P1) and has one 4-pin I²C header (P2) that goes to the host. The I²C lines idle at 5 V, so whatever receives the data needs to be 5 V tolerant. Also, the 4.7 kΩ pull-ups on this board (R18, R19) end up in parallel with any other pull-ups on the bus.

For power, the 5 V rail has to supply roughly 15 mA of sensor bias current (16 channels at about 0.94 mA each), about 9 mA for the fault LED (assuming a forward drop around 2 V), and whatever the microcontroller pulls. That is easily inside the regulator's 0.5 A rating. On the data side, each cycle sends 17 bytes, which fits in the 32-byte Arduino `Wire` buffer.

5 Notes/Findings

5.1 Core Requirements

The core functionality of the temperature monitor is

1. to run from 12 V and make its own regulated 5 V rail
2. to read 16 diode sensors, one at a time, through a single analog multiplexer and one ADC pin
3. to send the maximum temperature and all 16 temperatures to address 0x48 over I²C once per cycle
4. to blink the fault LED when any temperature is above 60 °C and keep it off otherwise

(a) it follows that the firmware has to track the maximum of all 16 readings every cycle

5.2 How the Sensing Works

Each sensor is just a diode. When you push a steady current through a silicon diode, the voltage across it (the forward voltage) drops as it gets warmer, by roughly 2 mV for every 1 °C. At 25 °C it sits around 0.6 V. So if you measure that voltage, you can work backwards to temperature.

On this board every channel is the same simple circuit (Figure 1): a 4.7 kΩ resistor from +5 V to a node, and a diode from that node to ground. The node goes to one input on the multiplexer. At 25 °C the bias current works out to

$$I = (5 - 0.6) / 4700 \approx 0.94\ mA$$

and the node voltage just follows the diode's forward voltage. The resistor dissipates about 4.1 mW and the diode about 0.56 mW, so self-heating isn't something to worry about.

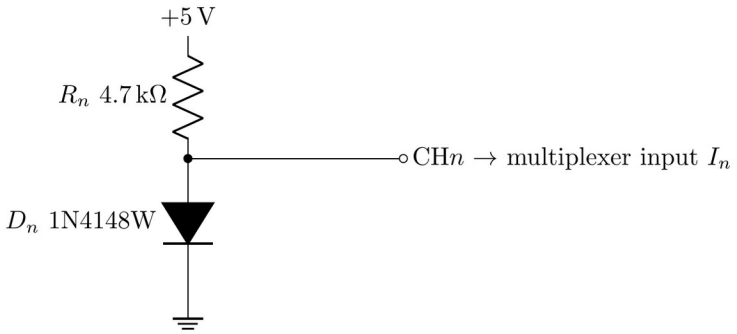


Figure 1: One sensing channel (repeated 16 times, n = 0 to 15).

5.3 Part Choices and Justification

The microcontroller (MD1) is an Arduino Nano-style module, part number A000005, which is the Arduino Nano Every. It runs at 5 V, has a 10-bit ADC and a hardware I²C port, and the Arduino framework keeps the firmware short. Its 5 V pin is fed straight from the board's +5 rail, and its 3.3 V pin, VIN pin, and RESET pins are left unconnected.

The CD74HC4067M96 (U1) is a 16-channel analog multiplexer. Four select lines (S0 to S3) pick one of the sixteen inputs and connect it to the common pin, which goes to ADC input A0. The enable pin (E) is active low and is driven by D6, so the firmware can switch the multiplexer on at start-up.

The R-78E5.0-0.5 (U2) is a switching regulator module with a 5 V, 500 mA output and a 7 to 28 V input range. It drops into the same kind of three-pin footprint as a linear regulator, but it doesn't burn the 12 V to 5 V drop off as heat. C2 and C3 (10 µF each) decouple its input and output.

Each sensor is a 1N4148W-7-F small-signal diode (D1 to D16), biased by a 4.7 kΩ resistor (R2 to R17) to +5 V as described in section 5.2.

Reference	Part	Function
MD1	Arduino Nano Every (A000005)	Microcontroller, ADC, I²C master
U1	CD74HC4067M96	16-channel analog multiplexer
U2	R-78E5.0-0.5	12 V to 5 V switching regulator, 0.5 A

D1–D16	1N4148W-7-F	Temperature sensing diodes
R2–R17	4.7 kΩ	Sensor bias resistors
R18, R19	4.7 kΩ	I²C pull-ups (SDA, SCL)
R1	330 Ω	Fault LED series resistor
DS1	LED, 304090041	Fault indicator
C1	100 nF	Multiplexer supply decoupling
C2, C3	10 µF	Regulator input and output capacitors
P1	B2P-VH(LF)(SN)	12 V power input, 2-pin
P2	S4B-XH-A(LF)(SN)	I²C output header, 4-pin

Table 1: Bill of materials summary.

5.4 Schematic

Figure 2 is the full schematic. It breaks down into six blocks: the microcontroller (MD1), the multiplexer (U1), the power section (P1, U2, C2, C3), the fault LED (R1, DS1), the I²C header with its pull-ups (P2, R18, R19), and the sixteen-channel sensor array (R2 to R17, D1 to D16).

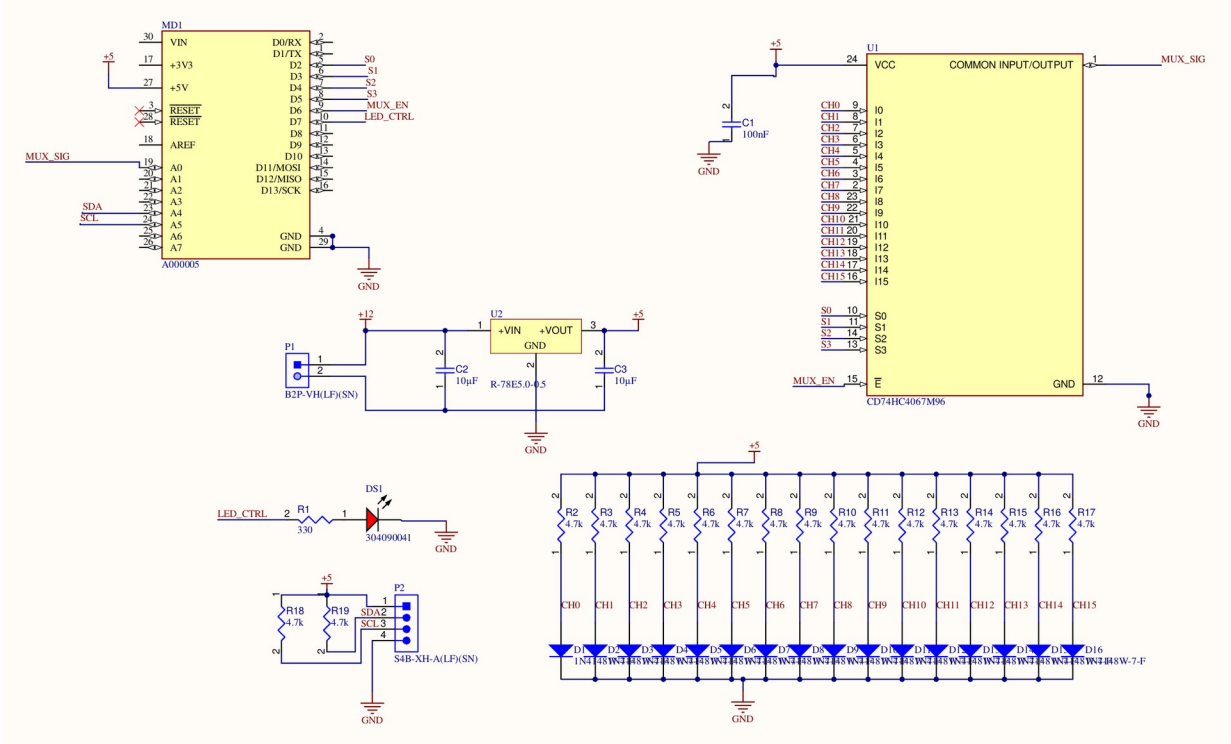


Figure 2: Schematic.

5.4.1 Pin Map

Table 2 shows how MD1 connects to the rest of the board.

Net	MD1 pin	Connects to	Direction
S0	D2	U1 pin 10 (S0)	Output

S1	D3	U1 pin 11 (S1)	Output
S2	D4	U1 pin 14 (S2)	Output
S3	D5	U1 pin 13 (S3)	Output
MUX_EN	D6	U1 pin 15 (E, active low)	Output
LED_CTRL	D7	R1, then DS1	Output
MUX_SIG	A0	U1 pin 1 (common)	Input
SDA	A4	P2 pin 2, R19	I²C
SCL	A5	P2 pin 3, R18	I²C

Table 2: Microcontroller pin map.

5.5 Workflow

Figure 3 shows how power and signals move through the board. Orange paths carry power and the dark ones carry signals. Here is the sequence from power-up to output:

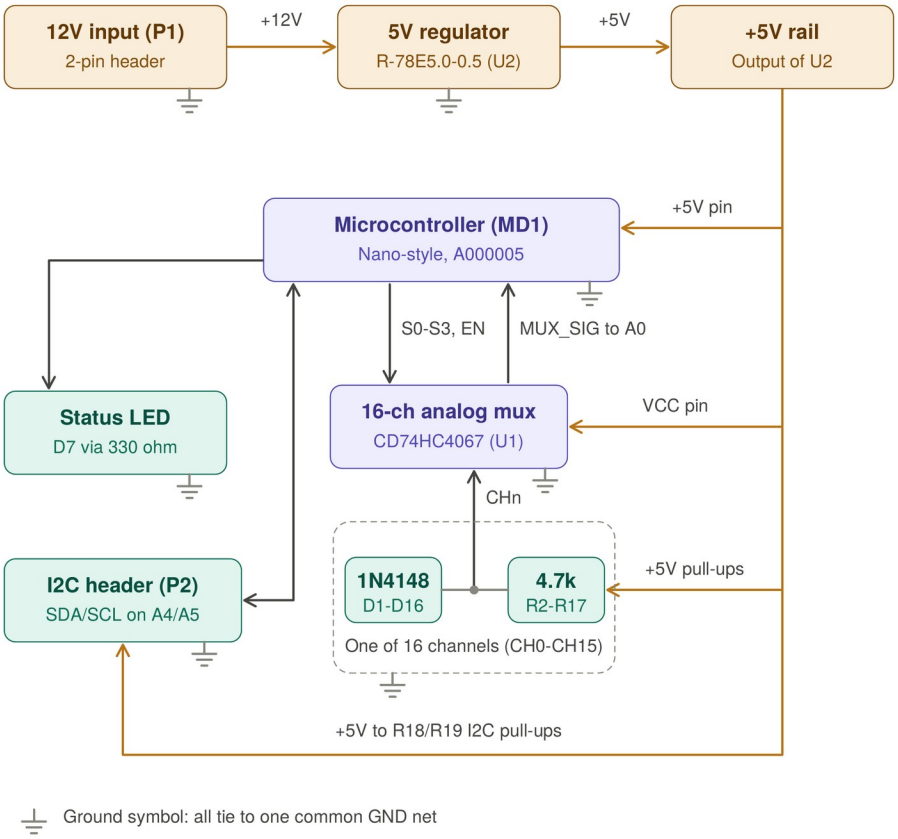


Figure 3: System flow chart.

1. The 12 V input (P1) feeds the regulator (U2), which makes the +5 rail.
2. The +5 rail powers the microcontroller (5 V pin), the multiplexer (VCC pin), the 16 sensor bias resistors, and the I²C pull-ups.
3. The microcontroller drives the select lines S0 to S3 and the enable line to pick a channel.

4. The selected sensor voltage (CHn) passes through the multiplexer to MUX_SIG, which goes into ADC input A0.
5. The firmware converts the reading into a temperature, then repeats for all 16 channels.
6. The results go out over the I²C header (SDA on A4, SCL on A5), and the status LED on D7 gets updated.

5.6 Firmware

The firmware is one Arduino sketch, `temp_monitor.ino`, written in C++ with the Arduino framework. It has a `setup()` function that runs once and a `loop()` function that repeats forever. Figure 4 (after the listings) shows one pass through `loop()`. The line numbers in the listings match the source file.

5.6.1 Pins and Constants

The top of the file names the pins from Table 2 and sets two settings: the I²C address the data goes to, and the fault temperature. The `temps` array holds the latest temperature for each channel. Below that are the two calibration points and the `voltageToTemp()` function, which converts a voltage to a temperature using linear interpolation (see 5.6.3).

```
// RFR Temperature Monitor - Group 21
// Reads 16 diode temp sensors through a mux, sends them over I2C,
// and blinks the fault LED if anything gets too hot

#include <Wire.h>

// Pin setup
const int S0 = 2;       // Mux select lines
const int S1 = 3;
const int S2 = 4;
const int S3 = 5;
const int MUX_EN = 6;   // Mux enable, active low
const int LED = 7;      // Fault LED
const int MUX_SIG = A0; // Mux output to ADC

const int I2C_ADDRESS = 0x48;  // Where the temps get sent
const float FAULT_TEMP = 60.0; // LED blinks above this temp (C)

float temps[16]; // Latest temp for each channel

// Calibration points (temp in C, voltage in V)
// Placeholders based on 0.6 V at 25 C and -2 mV/C, replace after testing
const float T1 = 25.0;
const float V1 = 0.600;
const float T2 = 75.0;
const float V2 = 0.500;

// Convert diode voltage to temp using linear interpolation between the two points
float voltageToTemp(float voltage) {
  return T1 + (voltage - V1) * (T2 - T1) / (V2 - V1);
}
```

5.6.2 setup()

`setup()` sets the six control pins as outputs, pulls the enable pin low to turn the multiplexer on (it's active low), and starts the I²C bus as a master. It also opens the serial port at 9600 baud so the readings can be checked from a laptop during testing.

```
33 void setup() {
34   pinMode(S0, OUTPUT);
35   pinMode(S1, OUTPUT);
36   pinMode(S2, OUTPUT);
37   pinMode(S3, OUTPUT);
38   pinMode(MUX_EN, OUTPUT);
39   pinMode(LED, OUTPUT);
40
41   digitalWrite(MUX_EN, LOW); // Turn on the mux
42
43   Wire.begin();       // Start I2C
44   Serial.begin(9600); // For checking readings
45 }
```

5.6.3 Reading the Channels

`loop()` starts by resetting `maxTemp`, then goes through each channel. The four low bits of the channel number get written to S0 to S3, a 1 ms delay gives the analog path time to settle, and `analogRead()` gives back a 10-bit count. That count gets converted to a voltage, and the voltage to a temperature:

$$V = (5.0 \times raw) / 1023 \qquad T = T_1 + (V - V_1) \times (T_2 - T_1) / (V_2 - V_1) \quad [°C]$$

This is linear interpolation: the firmware draws a straight line through two known points, (T1, V1) and (T2, V2), and reads the temperature off that line. The current points, 0.600 V at 25 °C and 0.500 V at 75 °C, are placeholders taken from the assumption in section 5.2 (0.6 V at 25 °C, dropping 2 mV per °C), so for now the result is the same as the original formula. Once the board is tested, they get replaced with measured values, which also takes out the offset from the multiplexer's on-resistance.

```
47 void loop() {
48   float maxTemp = -1000;
49
50   for (int ch = 0; ch < 16; ch++) {
51     // Select the channel on the mux
52     digitalWrite(S0, ch & 1);
53     digitalWrite(S1, (ch >> 1) & 1);
54     digitalWrite(S2, (ch >> 2) & 1);
55     digitalWrite(S3, (ch >> 3) & 1);
56     delay(1); // Let the signal settle
57
58     // Read the ADC and convert to voltage
59     int raw = analogRead(MUX_SIG);
60     float voltage = raw * 5.0 / 1023.0;
61
62     // Convert to temp
63     temps[ch] = voltageToTemp(voltage);
```

Each channel's voltage and temperature are then printed to the Serial Monitor so the calibration values can be read off during testing, and the highest temperature seen gets saved in `maxTemp`.

```
65      // Print voltage and temp for testing
66      Serial.print("CH");
67      Serial.print(ch);
68      Serial.print(": ");
69      Serial.print(voltage, 3);
70      Serial.print(" V, ");
71      Serial.print(temps[ch], 1);
72      Serial.println(" C");
73
74      // Track the highest temp
75      if (temps[ch] > maxTemp) {
76        maxTemp = temps[ch];
77      }
78    }
```

One ADC step is 5/1023 = 4.89 mV, which works out to about 2.44 °C per count. Table 3 shows what a few counts turn into with the current calibration points.

ADC count	Voltage (V)	Temperature (°C)	Meaning
123	0.601	24.4	Near room temperature
109	0.533	58.6	Highest count that does not trip the fault
108	0.528	61.1	Lowest count that trips the fault
0	0.000	325	Sensor shorted to ground
1023	5.000	−2175	Sensor open (pulled up to 5 V)

Table 3: Example ADC counts and the temperatures the firmware calculates from them.

5.6.4 Sending the Data and the Fault LED

After the scan, the sketch starts an I²C transmission to address 0x48 and writes the maximum temperature first, then the 16 channel temperatures. Each value gets cast to `int` and written as a single byte.

```
80    // Send max temp first, then all 16
81    Wire.beginTransmission(I2C_ADDRESS);
82    Wire.write((int)maxTemp);
83    for (int ch = 0; ch < 16; ch++) {
84      Wire.write((int)temps[ch]);
85    }
86    Wire.endTransmission();
```

Last, it compares `maxTemp` to `FAULT_TEMP`. While the limit is exceeded, the LED toggles once per pass. Otherwise it’s forced off. The pass ends with a 500 ms delay.

```
88    // Blink fault LED if any sensor is over the limit, otherwise keep it off
89    if (maxTemp > FAULT_TEMP) {
90      digitalWrite(LED, !digitalRead(LED));
91    } else {
92      digitalWrite(LED, LOW);
93    }
94
95    delay(500);
96  }
```

5.6.5 Timing

The timing comes straight from the code. The scan takes about 16 ms because of the sixteen 1 ms settling delays, plus the ADC conversions. Sending 18 bytes (the address plus 17 data bytes) at the standard 100 kHz I²C rate takes about 1.6 ms. The serial printout is about 360 characters per pass, which takes roughly 0.3 s at 9600 baud. Add the 500 ms delay and one pass takes roughly 0.8 s, so readings update a little more than once a second. The LED toggles once per pass, so it blinks with a period of about 1.6 s. Removing the serial prints after calibration brings the pass back to about 0.52 s.

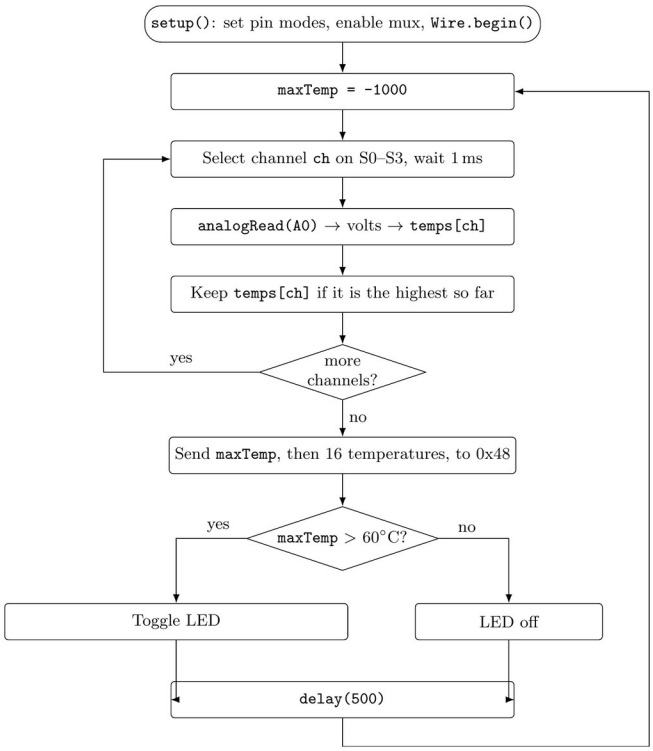


Figure 4: Firmware flow for one pass of loop().

Quantity	Value
Sensor bias current (per channel)	≈ 0.94 mA
Total sensor bias current (16 channels)	≈ 15 mA
ADC step	4.89 mV
Temperature per ADC count	≈ 2.44 °C
Fault trips at ADC count	≤ 108
I²C payload per pass	17 bytes
Loop period	≈ 0.8 s

Table 4: Key design numbers (calculated by hand from the schematic and firmware).

5.7 Limitations and Recommendations

1. Double-check the 12 V return. In the exported schematic, the wire from P1 pin 2 to the bottoms of C2 and C3 crosses the U2 ground stub with no junction dot, and the exported net info lists those three pins as separate from the GND net. Worth checking in the schematic editor to make sure the input return really is tied to GND.
2. Calibrate each channel. The conversion assumes exactly 0.6 V at 25 °C for every diode, but they vary a bit. Measuring each channel at a known temperature and storing an offset would fix that.
3. Catch open and shorted sensors. A count near 0 or near 1023 means a shorted or open sensor (Table 3). The firmware could flag those separately instead of treating them like a real temperature.
4. Clamp the value before sending. The I²C byte is a single 8-bit value, so anything outside −128 to 127 °C wraps around. An open sensor (computed −2175 °C) goes out as the byte 129, for example. Clamping first, and making sure the receiver reads signed bytes, avoids that.
5. Add some hysteresis to the fault. If a reading hovers right around 60 °C, the LED will flip between blinking and off every pass. A small buffer around the threshold would make it steadier.
6. Be careful with USB and 12 V at the same time. The microcontroller’s 5 V pin is fed from the board’s +5 rail, so it’s worth checking the module’s schematic for back-feeding before powering from 12 V while plugged into USB.

6 Conclusion

The repository documents a 16-channel diode-based temperature monitor using one analog multiplexer and one ADC input. The Arduino sketch is designed to convert the readings to temperatures, transmit the maximum and individual values over I²C to address 0x48, and blink a fault LED when the measured maximum exceeds 60 °C. The design is supported by hardware schematics and a firmware workflow, but practical validation is still needed. Before testing, the missing closing brace in the committed sketch should be fixed, and sensor calibration, open/short detection, and byte-range handling should be reviewed.

Source: RFR27 New Member Project, Group 21 — https://github.com/Rdek113/rfr27-newmember-21